

第2章 基于特征点法的视觉里程计

CONTENTS PAGE

2.1 绪论

2.2 特征点提取与匹配

2.3 2D-2D对极几何

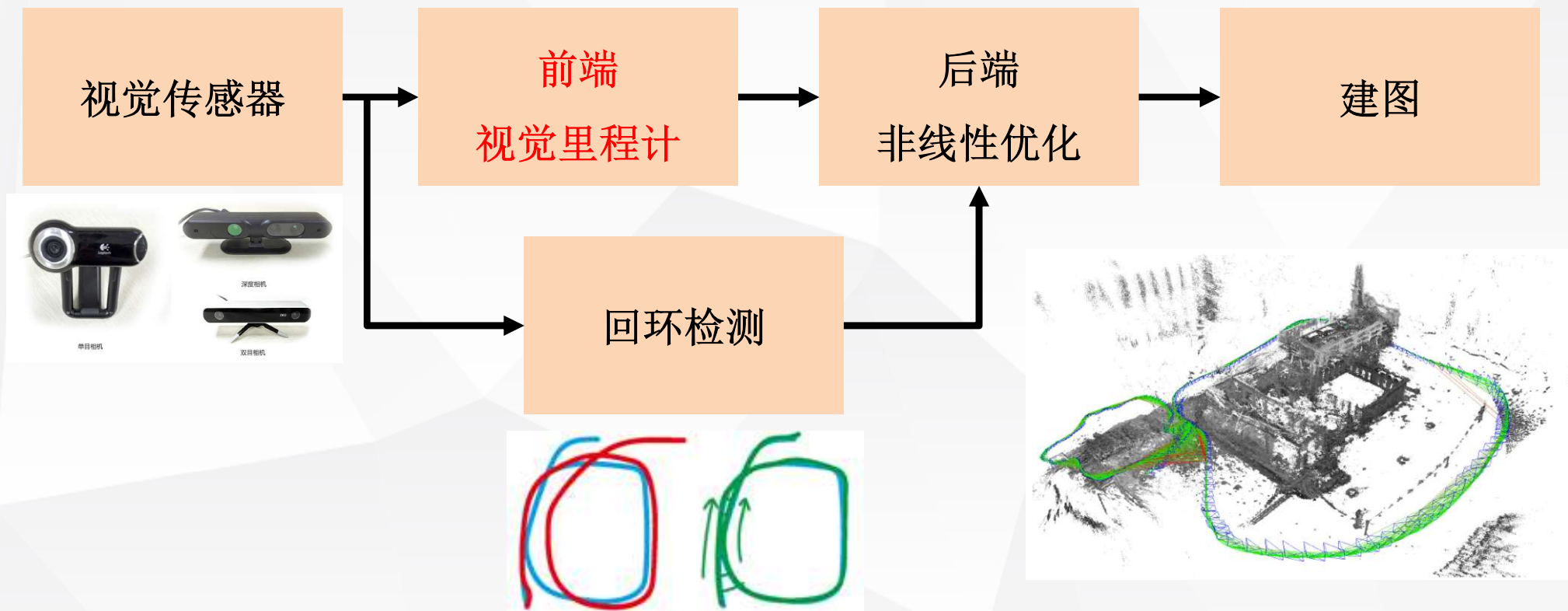
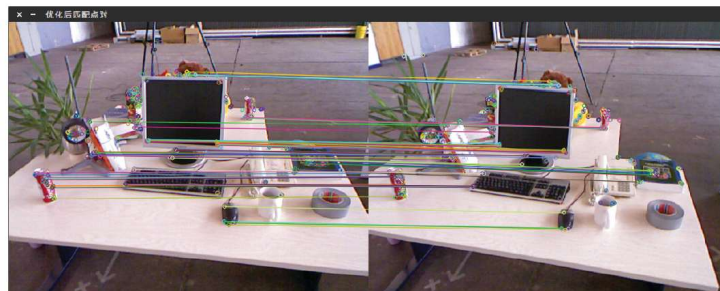
2.4 3D-2D PNP

2.5 3D-3D ICP

2.6 小结

2.1 绪论

➤ 视觉SLAM框架结构



2.1 绪论

➤ 里程计（Odometry）：

完成运动估计的装置（包括硬件和算法）

➤ 视觉里程计（Visual Odometry, 简称VO）：

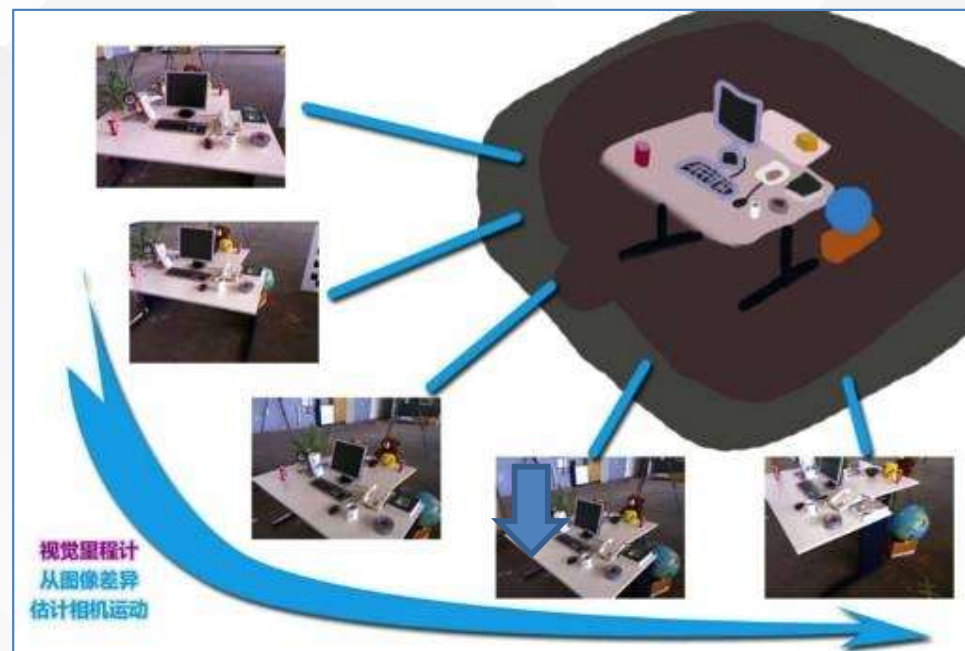
主要依靠视觉传感器（比如单目、双目相机）的里程计，仅利用单个或多个相机的输入信息估计智能体的运动信息。

➤ 视觉里程计研究的问题：

如何从几个相邻图像中，估计相机的运动。通过获取相邻两帧之间的 RT 变换关系（旋转和平移），将获取得到的多个 RT 相乘获取得到当前帧与原始位置之间的变换关系，然后进行迭代优化。

2.1 绪论

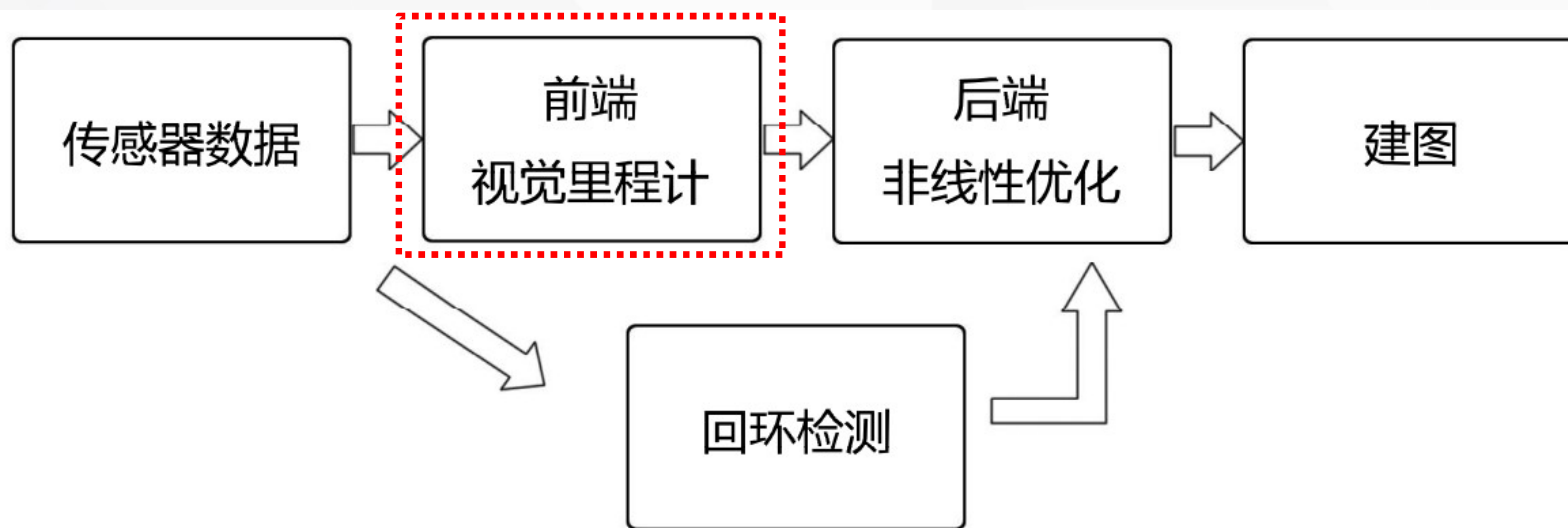
- 具体操作：将一个摄像头（或多个摄像头）刚性连接到一个移动的物体上（如机器人），通过摄像头采集的视频流来确定相机的6自由度：



- 单目视觉里程计：使用1个摄像头；
- 立体视觉里程计：使用两个（或者更多）摄像头

2.1 绪论

- 视觉里程计（VO）是视觉SLAM的前端，根据相邻图像的信息，估计出相机运动，给后端提供较好的初始值。
- 按照是否需要提取特征：特征点法的前端和直接法前端（不提取特征）。
- 特征点法的前端：到目前为止被认为是VO的主流方法。运行稳定，是目前比较成熟的解决方案。



视觉SLAM框架

第2章 基于特征点法的视觉里程计

CONTENTS PAGE

2.1 绪论

2.2 特征点提取与匹配

2.3 2D-2D对极几何

2.4 3D-2D PNP

2.5 3D-3D ICP

2.6 小结

2.2 特征点提取与匹配

➤ 2.2.1 特征点提取

➤ 2.2.2 特征点匹配

2.2 特征点提取与匹配

➤ VO的主要任务：如何根据图像来估计相机运动

- ① 从图像中选取比较有代表性的点：这些点在相机视角发生少量变化后会保持不变，在各个图像中找到相同的点。
- ② 在这些点的基础上，研究相机位姿估计问题以及这些点的定位问题。
- ③ 在经典SLAM模型中，这些具有代表性的点称为路标。

- ✓ 路标是三维空间中固定不变的点，能够在特定位姿下观测到；
- ✓ 数量充足，以实现良好的定位；
- ✓ 较好的区分性，以实现数据关联。

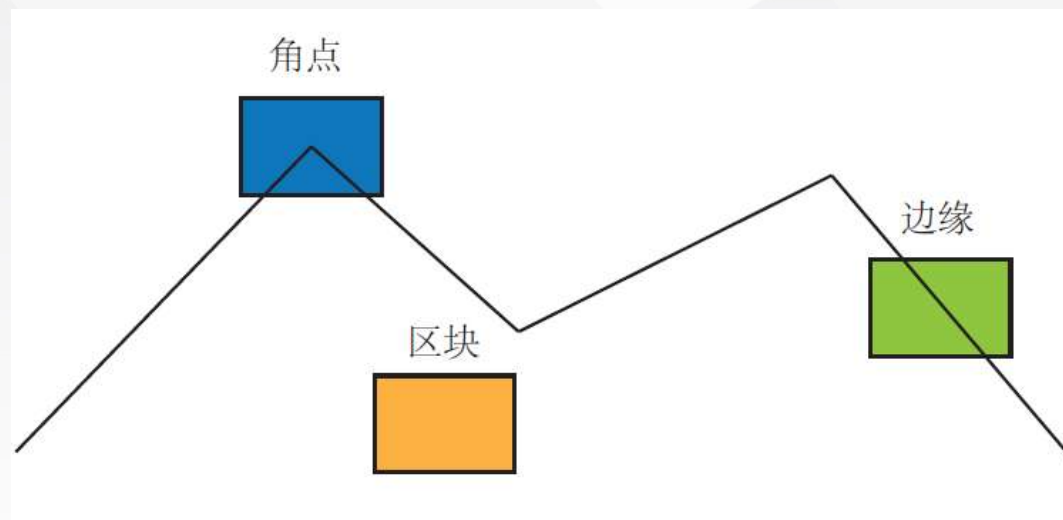
➤在视觉SLAM算法中，路标指的是图像特征（feature）。



2.2.1 特征点提取

➤ 特征点

图像当中具有代表性的部分。在相机视角发生少量变化后保持不变，因此会在各个图像上找到相同的点。



- **可重复性** (Repeatability)：相同的“区域”可以在不同的图像中找到；
- **可区别性** (Distinctiveness)：不同的“区域”具有不同的表达；
- **高效率** (Efficiency)：同一图像中，特征点的数量应远小于像素的数量；
- **本地性** (Locality)：特征仅与一小片图像区域相关。

2.2.1 特征点提取

➤ 特征点的信息

提取关键点，并计算描述子

① 关键点（key-point）：

主要指的是该特征点在图像中的位置，有的特征点还具有朝向、大小等信息；

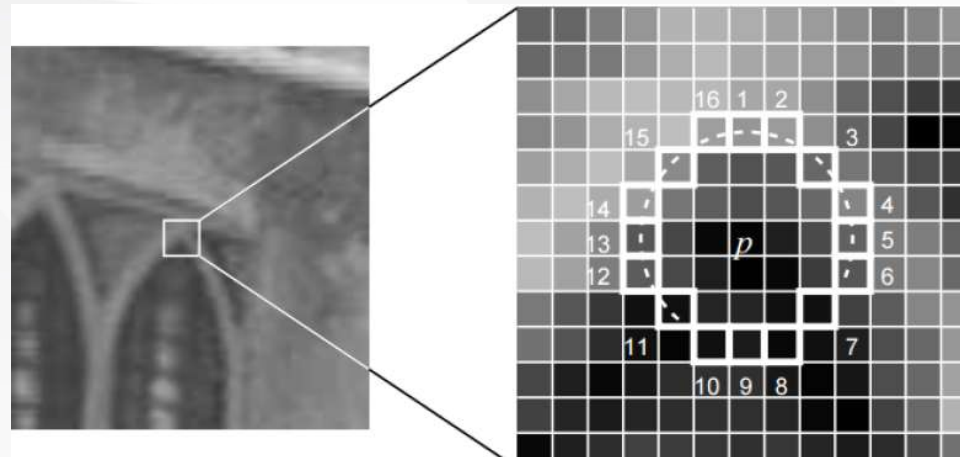
② 描述子（Descriptor）：

- 通常是一个向量，按照人为设计的方式，描述该关键点周围像素的信息。
- 按照“外观相似的特征应该具有相似的描述子”的原则设计。
- 只要两个特征点的描述子在向量空间上的距离相近，就可以认为它们是同样的特征点。

2.2.1 特征点提取

① FAST特征

- FAST仅有关键点，没有描述子；
- 基本思想：如果一个像素与邻域像素差别较大（过亮或过暗），该点可能是角点。FAST只需要比较像素亮度的大小，计算速度快。



适当降低精度和健壮性，提升计算速度

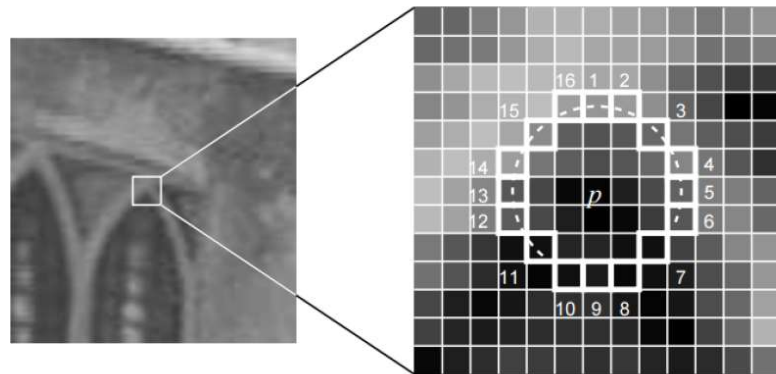
◆检测流程：

1. 在图像中选取像素 p ，假设它的亮度为 I_p
2. 设置一个阈值 T (比如 I_p 的 20%)。
3. 以像素 p 为中心，选取半径为 3 的圆上的 16 个像素点。
4. 假如选取的圆上，有连续的 N 个点的亮度大于 $I_p + T$ 或小于 $I_p - T$ ，那么像素 p 可以被认为是特征点 (N 通常取 12，即为 FAST-12。其它常用的 N 取值为 9 和 11，他们分别被称为 FAST-9, FAST-11)。
5. 循环以上四步，对每一个像素执行相同的操作。

2.2.1 特征点提取

① FAST特征

问题分析：



- ① FAST特征点数量很大且不确定，实际往往希望对图像提取固定数量的特征。
- ② FAST角点不具有尺度和方向信息。而且由于它固定选取半径为3的圆，存在尺度问题。



针对问题

② ORB (Oriented FAST and Rotated BRIEF) 特征

- 视觉SLAM常用的特征，被用于例如：ORB-SLAM系列框架
- 具有局部不变性，对FAST特征点与BRIEF特征描述子的一种结合与改进。在2011年一篇名为“ORB: An Efficient Alternative to SIFT or SURF”的文章中提出。
- ORB是除了SIFT与SURF外很好的选择，具有很高的效率。

2.2.1 特征点提取

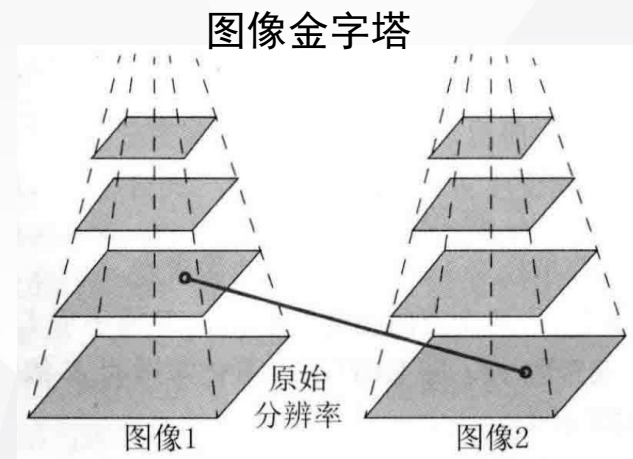
② ORB (Oriented FAST and Rotated BRIEF) 特征

- **关键点**: Oriented FAST, 一种改进的FAST角点
- **描述子**: BRIEF (Binary Robust Independent Elementary Feature)

a) 关键点: Oriented FAST

采用Oriented FAST对FAST进行改进, 可以描述角点的尺度与旋转特性:

- A. **在尺度方面**: ORB算法通过构建图像金字塔, 并在金字塔的每一层上检测FAST角点来实现FAST的**尺度不变性**;
- B. **在旋转方面**: 用灰度质心法 (通过计算特征点附近的图像灰度质心) 来实现FAST的**旋转不变性**。



2.2.1 特征点提取

② ORB (Oriented FAST and Rotated BRIEF) 特征

➤ 灰度质心法 (Intensity Centroid) :

对于任意一个特征点 p ，以该点为中心，选取一个小的图像块 B ：

1. 在一个小的图像块 B 中，定义图像块的矩为：

$$m_{pq} = \sum_{x,y \in B} x^p y^q I(x,y), \quad p, q = \{0, 1\}.$$

2. 通过矩可以找到图像块的质心：

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right).$$

3. 连接图像块的几何中心 O 与质心 C ，得到一个方向向量 $\overrightarrow{OC}$ ，于是特征点的方向可以定义为：

$$\theta = \arctan(m_{01}/m_{10}).$$

此时，FAST角点具备了方向信息，可实现旋转不变性。

2.2.1 特征点提取

② ORB (Oriented FAST and Rotated BRIEF) 特征

b) 描述子: BRIEF描述子

- ✓ 对前一步提取出特征点的周围图像区域进行描述;
- ✓ 原始BRIEF没有旋转不变性, ORB给BRIEF加上旋转不变性, 称为“Steer BRIEF”;

- 二进制描述子 (描述向量由许多1和0组成), 编码了关键点附近两个随机像素 (例如p和q) 的大小关系: 如果p比q大, 则取1, 反之取0; 如果取了128个这样的p和q, 则就得到了128维由0和1组成的向量。
- p和q的选取方法: 按照某种概率分布, 随机地挑选p和q的位置, 速度非常快, 适用于实时的图像匹配。
- 原始BRIEF描述子不具有旋转不变性, 在图像发生旋转时容易丢失。ORB在FAST特征点提取阶段计算了关键点的方向, 利用该方向信息计算旋转之后的BRIEF特征, 使得ORB的描述子具有较好的旋转不变性。

2.2 特征点提取与匹配

➤ 2.2.1 特征点提取

➤ 2.2.2 特征点匹配

2.2.2 特征点匹配

- 特征匹配解决SLAM中的**数据关联问题**，即确定当前看到的路标与之前看到的路标之间的对应关系。



- 图像匹配过程：在图像 I_t 中提取到特征点 x_t^m ，在图像 I_{t+1} 中提取到特征点 x_{t+1}^n ，特征匹配方法是**暴力匹配（Brute-Force Matcher）**。计算每一个特征点 x_t^m 与所有的 x_{t+1}^n **测量描述子的距离**，然后排序，取最近的一个作为匹配点。

2.2.2 特征点匹配

- **描述子距离**：表示了两个特征之间的相似程度，可以取不同的距离度量范数。

- ① 对于**浮点类型的描述子**，可以采用**欧氏距离**进行度量。
- ② 对于**二进制的描述子（比如BRIEF）**，使用**汉明距离**作为度量：
两个二进制串之间的汉明距离，指的是其**不同位数的个数**。

- 特征点数量很大时，暴力匹配法的运算量很大。
- **快速近似最近邻**（FLANN, Fast Library for Approximate Nearest Neighbors）算法适用于匹配点数量极多的情况。

2.2.2 特征点匹配

■ OpenCV 开源图像库



- 输入两张图像
- 操作：ORB特征提取、特征匹配

2.2.2 特征点匹配

//-- 读取图像

```
Mat img_1 = imread ( argv[1], CV_LOAD_IMAGE_COLOR );
```

```
Mat img_2 = imread ( argv[2], CV_LOAD_IMAGE_COLOR );
```

//-- 初始化

```
std::vector<KeyPoint> keypoints_1, keypoints_2;
```

```
Mat descriptors_1, descriptors_2;
```

```
Ptr<ORB> orb = ORB::create ( 500, 1.2f, 8, 31, 0, 2, ORB::HARRIS_SCORE,31,20 );
```

//-- 第一步：检测 *Oriented FAST* 角点位置

```
orb->detect ( img_1, keypoints_1 );
```

```
orb->detect ( img_2, keypoints_2 );
```

//-- 第二步：根据角点位置计算 *BRIEF* 描述子

```
orb->compute ( img_1, keypoints_1, descriptors_1 );
```

```
orb->compute ( img_2, keypoints_2, descriptors_2 );
```



ORB关键点

//-- 第三步：对两幅图像中的**BRIEF**描述子进行匹配，使用 *Hamming* 距离

```
vector<DMatch> matches;
```

```
BFMatcher matcher ( NORM_HAMMING );
```

```
matcher.match ( descriptors_1, descriptors_2, matches );
```

//-- 第四步：匹配点对筛选

```
double min_dist=10000, max_dist=0;
```

// 找出所有匹配之间的最小距离和最大距离，即是最相似的和最不相似的两组点之间的距离

```
for ( int i = 0; i < descriptors_1.rows; i++ )
```

```
{
```

```
    double dist = matches[i].distance;
```

```
    if ( dist < min_dist ) min_dist = dist;
```

```
    if ( dist > max_dist ) max_dist = dist;
```

```
}
```

```
printf ( "-- Max dist : %f \n", max_dist );
```

```
printf ( "-- Min dist : %f \n", min_dist );
```

// 当描述子之间的距离大于两倍的最小距离时，即认为匹配有误。

// 但有时候最小距离会非常小，设置一个经验值作为下限。

```
std::vector< DMatch > good_matches;
```

```
for ( int i = 0; i < descriptors_1.rows; i++ )
```

```
{
```

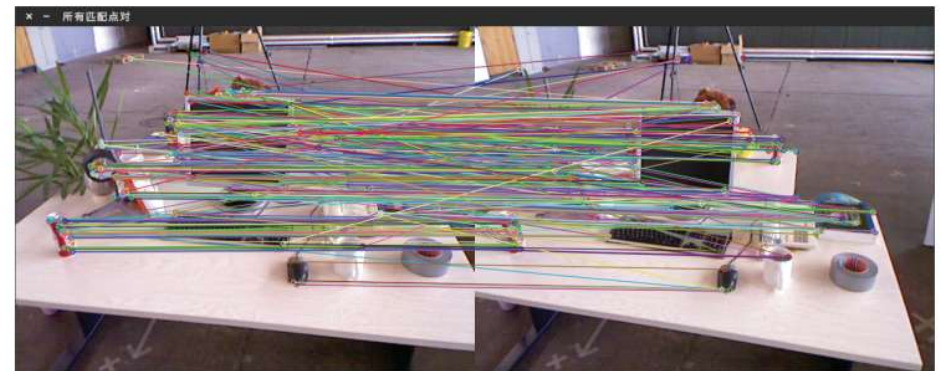
```
    if ( matches[i].distance <= max ( 2*min_dist, 30.0 )
```

```
    {
```

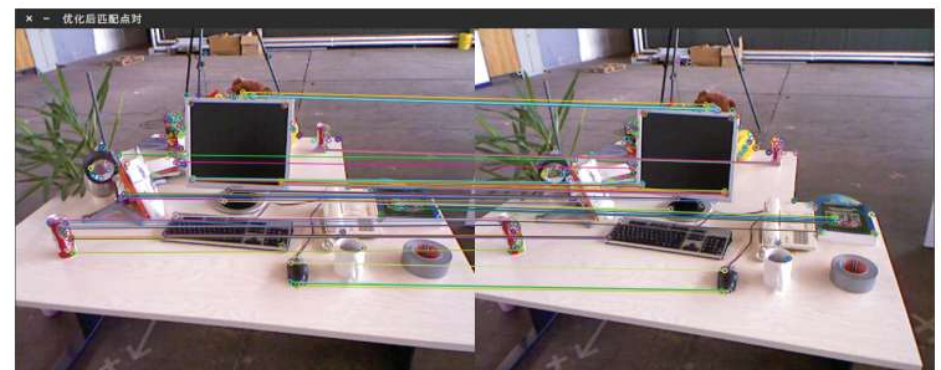
```
        good_matches.push_back ( matches[i] );
```

```
    }
```

```
}
```



未筛选的匹配



筛选后的匹配

2.2.2 特征点匹配

➤ 根据匹配的点对估计相机的运动，由于相机的原理不同，分如下情况：

- ① 相机为单目时，只知道2D的像素坐标，因此问题是根据两组2D点估计相机运动。解决方法：对极几何（2D-2D）。
- ② 相机为双目、RGB-D时，或者通过某种方法得到了距离信息，问题就是根据两组3D点估计相机运动。解决方法：ICP（3D-3D）。
- ③ 如果有3D点和它们在相机的投影位置，也能估计相机运动。解决方法：PnP（3D-2D）。



第2章 基于特征点法的视觉里程计

CONTENTS PAGE

2.1 绪论

2.2 特征点提取与匹配

2.3 2D-2D对极几何

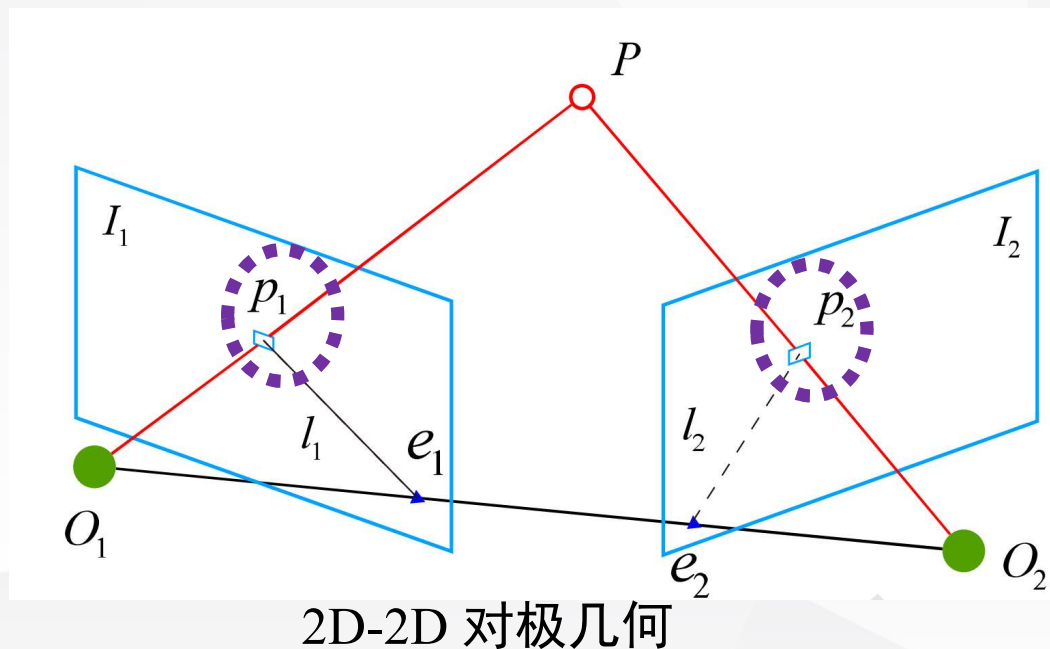
2.4 3D-2D PNP

2.5 3D-3D ICP

2.6 小结

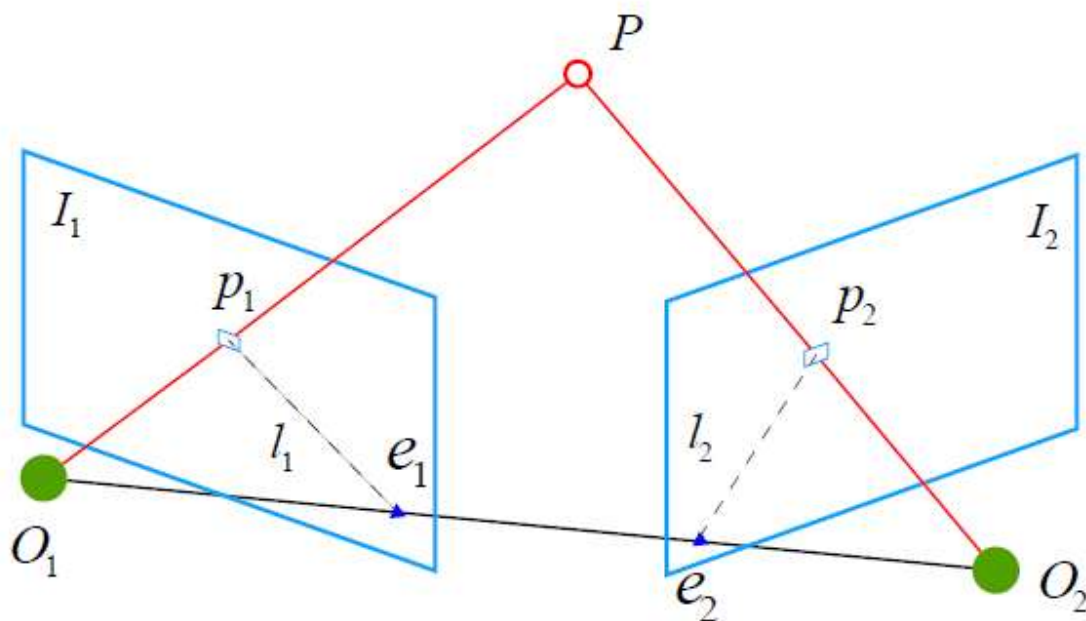
2.3 2D-2D对极几何

- **对极几何**：根据图像二维平面信息来估计单目相机帧间运动或双目相机相对位姿关系的一种算法；
- **研究出发点**：当相机在两个不同视角对同一物体进行拍摄时，物体在两幅图像中的成像不同，根据这两幅不同的图像，判断出相机位姿的变化。
- **已知条件**：两幅图像中匹配好的特征点对。
- **求解目标**：求取两帧图像之间的运动，假设第一帧到第二帧的运动为 R, t 。



- ✓ 假设从两张图像中得到了一对配对好的特征点 (p_1 和 p_2)。
- ✓ 如果有若干对这样的匹配点，可以通过这些二维图像点的对应关系，恢复出在两帧之间摄像机的运动。

2.3 2D-2D对极几何



O_1, O_2 : 相机中心.

p_1, p_2 : 同一个空间点 P 在两个成像平面上的投影。

O_1O_2P : 极平面

e_1, e_2 : 极点 (Epipoles)

l_1, l_2 : 极线 (Epipolar line)

O_1O_2 : 基线 (Baseline)

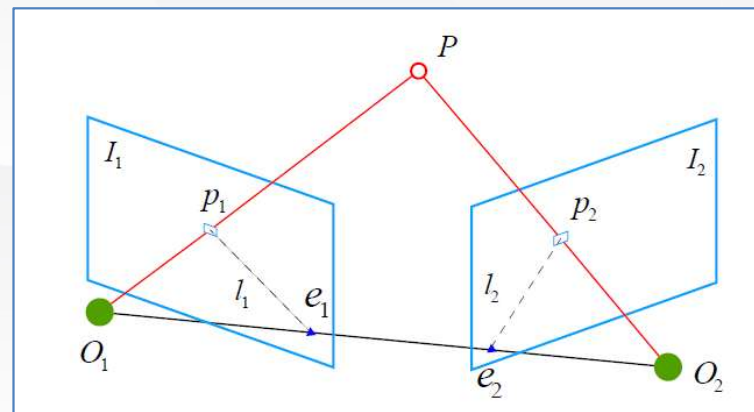
- 图中各符号之间的几何关系:

- ✓ 连线 $\overrightarrow{O_1p_1}$ 和 $\overrightarrow{O_2p_2}$ 在三维空间中会交于点 P 。 O_1, O_2, P 三个点确定一个平面，称为极平面 (Epipolar plane)
- ✓ O_1O_2 连线与像平面 I_1, I_2 的交点分别为极点 e_1, e_2 ;
- ✓ O_1O_2 称为基线;
- ✓ 极平面 (O_1O_2P) 与两个像平面 I_1, I_2 之间的相交线 l_1, l_2 称为极线

2.3 2D-2D对极几何

➤ 在第一帧的坐标系下，设 P 的空间位置（未知）为：

$$P = [X, Y, Z]^T$$



① 根据针孔相机模型，两个像素点 p_1, p_2 的像素位置为：

$$s_1 p_1 = K P, \quad s_2 p_2 = K (R P + t)$$

K 为相机内参矩阵；
 R, t 为两个坐标系的相对运动。

若采用齐次坐标表示像素点，在使用齐次坐标时，一个向量将等于它自身乘上任意的非零常数（齐次坐标意义下相等），这种相等关系也称为**尺度意义下相等**，记作：

$$sp \simeq p$$

此时，上述两个投影关系可写为：

$$p_1 \simeq K P, \quad p_2 \simeq K (R P + t)$$

② 现在，取： $x_1 = K^{-1} p_1, \quad x_2 = K^{-1} p_2$

这里的 x_1, x_2 是两个像点在相机归一化平面上的坐标，代入上述公式，得到：

$$x_2 \simeq R x_1 + t$$

2.3 2D-2D对极几何

$$\begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \mathbf{b} \triangleq \mathbf{a}^\wedge \mathbf{b}.$$

$$\mathbf{x}_2 \simeq \mathbf{R} \mathbf{x}_1 + \mathbf{t}$$

③ 两边同时左乘 $\mathbf{t}^\wedge$ ，这相当于同时与 $\mathbf{t}$ 做外积：

$$\mathbf{t}^\wedge \mathbf{x}_2 \simeq \mathbf{t}^\wedge \mathbf{R} \mathbf{x}_1$$

↓ 两侧同时左乘 $\mathbf{x}_2^T$

$$\mathbf{x}_2^T \mathbf{t}^\wedge \mathbf{x}_2 \simeq \mathbf{x}_2^T \mathbf{t}^\wedge \mathbf{R} \mathbf{x}_1$$

等式左侧， $\mathbf{t}^\wedge \mathbf{x}_2$ 是一个与 $\mathbf{t}$ 和 $\mathbf{x}_2$ 都垂直的向量。把它再和 $\mathbf{x}_2$ 做内积时，得到0。

$$\mathbf{x}_2^T \mathbf{t}^\wedge \mathbf{R} \mathbf{x}_1 = 0$$

↓ 重新代入 p_1, p_2 ,

$$\mathbf{x}_1 = \mathbf{K}^{-1} \mathbf{p}_1 \quad \mathbf{x}_2 = \mathbf{K}^{-1} \mathbf{p}_2$$

$$\mathbf{p}_2^T \mathbf{K}^{-T} \mathbf{t}^\wedge \mathbf{R} \mathbf{K}^{-1} \mathbf{p}_1 = 0.$$

把中间部分记作两个矩阵：基础矩阵 $\mathbf{F}$ 和本质矩阵 $\mathbf{E}$ ： $\mathbf{E} = \mathbf{t}^\wedge \mathbf{R}$, $\mathbf{F} = \mathbf{K}^{-T} \mathbf{E} \mathbf{K}^{-1}$

对极约束：

$$\mathbf{x}_2^T \mathbf{E} \mathbf{x}_1 = \mathbf{p}_2^T \mathbf{F} \mathbf{p}_1 = 0$$

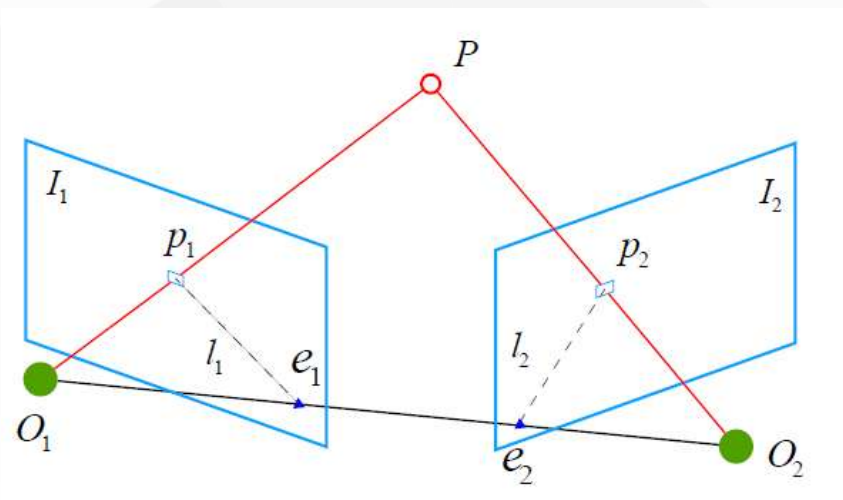
2.3 2D-2D对极几何

对极约束: $x_2^T E x_1 = p_2^T F p_1 = 0$ $E = t^\wedge R, \quad F = K^{-T} E K^{-1}$

➤ 对极约束给出了两个匹配点（ p_1 和 p_2 ）的空间位置关系。

➤ 于是，相机位姿估计问题变为以下两步：

1. 根据配对点的像素位置，求出 E 或者 F ；
2. 根据 E 或者 F ，求出 R, t 。



➤ 由于 E 和 F 只相差了相机内参，而内参在SLAM中通常是已知的，所以实践当中往往使用形式更简单的 E 。以 E 为例，介绍上面两个问题如何求解。

E 为 3×3 的矩阵，有9个未知数。 E 由对极约束定义且对极约束是等式为零的约束，对 E 乘以任意非零常数后，对极约束依然满足。表明： E 在不同尺度下等价。

2.3 2D-2D对极几何

$$\text{对极约束: } x_2^T E x_1 = p_2^T F p_1 = 0$$

$$x_1 = K^{-1} p_1, \quad x_2 = K^{-1} p_2$$

➤ 八点法求解 E

- 将 E 矩阵（3x3的矩阵），去掉尺度因子后剩八个自由度
- 已知一对匹配点，相机归一化平面坐标为： $x_1 = [u_1, v_1, 1]^T$, $x_2 = [u_2, v_2, 1]^T$ 。
- 根据对极约束，得到：

$$(u_2, v_2, 1) \begin{pmatrix} e_1 & e_2 & e_3 \\ e_4 & e_5 & e_6 \\ e_7 & e_8 & e_9 \end{pmatrix} \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix} = 0$$



将矩阵 E 展开，写成向量形式：

$$e = [e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9]^T$$

- 对极约束可写成与 e 有关的线性形式：

$$[u_2 u_1, u_2 v_1, u_2, v_2 u_1, v_2 v_1, v_2, u_1, v_1, 1] \cdot e = 0$$

2.3 2D-2D对极几何

➤ 八点法求解 E

- 八对点构成方程组

$$\begin{pmatrix} u_2^1 u_1^1 & u_2^1 v_1^1 & u_2^1 & v_2^1 u_1^1 & v_2^1 v_1^1 & v_2^1 & u_1^1 & v_1^1 & 1 \\ u_2^2 u_1^2 & u_2^2 v_1^2 & u_2^2 & v_2^2 u_1^2 & v_2^2 v_1^2 & v_2^2 & u_1^2 & v_1^2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_2^8 u_1^8 & u_2^8 v_1^8 & u_2^8 & v_2^8 u_1^8 & v_2^8 v_1^8 & v_2^8 & u_1^8 & v_1^8 & 1 \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \\ e_6 \\ e_7 \\ e_8 \\ e_9 \end{pmatrix} = 0.$$

系数矩阵由特征点位置构成

- 八个方程构成了一个线性方程组。它的系数矩阵由特征点位置构成，大小为8*9。

➤ 问题：如何根据已经估得的本质矩阵 E ，恢复出相机的运动 R ， t 。

$$E = t^\wedge R$$

奇异值分解
(SVD) 方法：

$$E = U \Sigma V^T$$



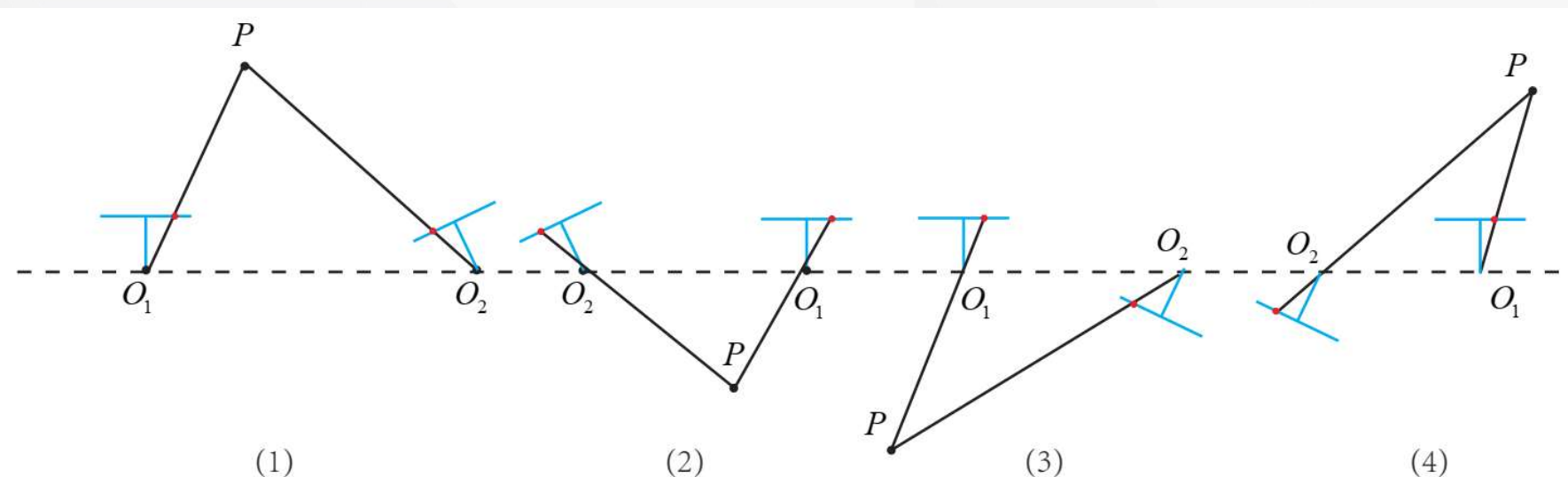
$$t_1^\wedge = U R_Z(\frac{\pi}{2}) \Sigma U^T, \quad R_1 = U R_Z^T(\frac{\pi}{2}) V^T$$

$$t_2^\wedge = U R_Z(-\frac{\pi}{2}) \Sigma U^T, \quad R_2 = U R_Z^T(-\frac{\pi}{2}) V^T.$$

得到4
组解

2.3 2D-2D对极几何

- 下图展示了分解本质矩阵 E 得到的4个解。已知空间点在相机（蓝色线）上的投影（红色点），要求解相机的运动。要保持红色点不变的情况下，可以有4种可能的情况。其中，只有第1种解中 P 在两个相机中均具有正的深度。因此，只需要把任意一点带入4种解中，检测该点在两个相机下的深度，就可以确定哪个解是正确的。



$$\begin{aligned} \hat{t}_1 &= UR_Z(\frac{\pi}{2})\Sigma U^T, & R_1 &= UR_Z^T(\frac{\pi}{2})V^T \\ \hat{t}_2 &= UR_Z(-\frac{\pi}{2})\Sigma U^T, & R_2 &= UR_Z^T(-\frac{\pi}{2})V^T. \end{aligned}$$

2.3 2D-2D对极几何

➤ 单应矩阵 H

- 除了基础矩阵 F 和本质矩阵 E ，还存在一种常见的矩阵：**单应矩阵**（Homography） H ，描述了两个平面之间的映射关系。

若场景中的特征点都落在同一平面上（比如墙、地面等），则可以通过单应性来进行运动估计。在无人机携带的俯视相机，或扫地机携带的顶视相机中比较常见。

- 单应矩阵**：描述处于共同平面上的一些点在两张图像之间的变换关系。考虑在图像 I_1 和 I_2 有一对匹配好的特征点 p_1 和 p_2 。这些**特征点位于平面 P 上**，设该平面 P 满足：

$$\boxed{n^T P + d = 0.} \xrightarrow{\text{整理得到}} \boxed{-\frac{n^T P}{d} = 1.}$$

2.3 2D-2D对极几何

➤ 单应矩阵 H

$$n^T P + d = 0.$$

整理得到

$$-\frac{n^T P}{d} = 1.$$

根据前面介绍, $p_2 \simeq K(RP + t)$ (尺度意义下相等), 对其进行进一步整理:

$$\begin{aligned} p_2 &\simeq K(RP + t) \\ &\simeq K \left(RP + t \cdot \left(-\frac{n^T P}{d} \right) \right) \\ &\simeq K \left(R - \frac{tn^T}{d} \right) \underline{P} \\ &\simeq K \left(R - \frac{tn^T}{d} \right) K^{-1} p_1. \end{aligned}$$

单应矩阵 H

$$\begin{aligned} p_1 &\simeq KP, \quad p_2 \simeq K(RP + t) \\ P &\simeq K^{-1} p_1 \end{aligned}$$

得到了直接描述图像坐标 p_1 和 p_2 之间的变换, 中间部分记为 H

$$p_2 \simeq Hp_1$$

2.3 2D-2D对极几何

$$n^T P + d = 0.$$

$$p_2 \simeq H p_1 \quad H = K \left(R - \frac{t n^T}{d} \right) K^{-1}$$

- H 与旋转 R 、平移 t 以及平面 P 的参数有关。单应矩阵 H 也是一个3*3的矩阵;
- H 求解思路: 根据匹配点计算 H , 将 $p_2 \simeq H p_1$ 展开:

$$\begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix} \simeq \begin{pmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{pmatrix} \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix}$$

这里的等号是尺度意义下相等, 所以 H 矩阵也可以乘以任意非零常数。在实际处理中令 $h_9=1$ 。然后根据第3行, 去掉这个非零因子, 得到:

$$u_2 = \frac{h_1 u_1 + h_2 v_1 + h_3}{h_7 u_1 + h_8 v_1 + h_9}$$

$$v_2 = \frac{h_4 u_1 + h_5 v_1 + h_6}{h_7 u_1 + h_8 v_1 + h_9}.$$

整理

令 $h_9=1$

$$h_1 u_1 + h_2 v_1 + h_3 - h_7 u_1 u_2 - h_8 v_1 u_2 = u_2$$

$$h_4 u_1 + h_5 v_1 + h_6 - h_7 u_1 v_2 - h_8 v_1 v_2 = v_2.$$

2.3 2D-2D对极几何

$$u_2 = \frac{h_1 u_1 + h_2 v_1 + h_3}{h_7 u_1 + h_8 v_1 + h_9}$$

$$v_2 = \frac{h_4 u_1 + h_5 v_1 + h_6}{h_7 u_1 + h_8 v_1 + h_9}$$

整理

令 $h_9=1$

$$h_1 u_1 + h_2 v_1 + h_3 - h_7 u_1 u_2 - h_8 v_1 u_2 = u_2$$

$$h_4 u_1 + h_5 v_1 + h_6 - h_7 u_1 v_2 - h_8 v_1 v_2 = v_2$$

- 一组匹配点对构造出两项约束；
- 因此，自由度为8的**单应矩阵** H 可以通过4对匹配特征点算出，即求解以下的线性方程组：

$$\begin{pmatrix} u_1^1 & v_1^1 & 1 & 0 & 0 & 0 & -u_1^1 u_2^1 & -v_1^1 u_2^1 \\ 0 & 0 & 0 & u_1^1 & v_1^1 & 1 & -u_1^1 v_2^1 & -v_1^1 v_2^1 \\ u_1^2 & v_1^2 & 1 & 0 & 0 & 0 & -u_1^2 u_2^2 & -v_1^2 u_2^2 \\ 0 & 0 & 0 & u_1^2 & v_1^2 & 1 & -u_1^2 v_2^2 & -v_1^2 v_2^2 \\ u_1^3 & v_1^3 & 1 & 0 & 0 & 0 & -u_1^3 u_2^3 & -v_1^3 u_2^3 \\ 0 & 0 & 0 & u_1^3 & v_1^3 & 1 & -u_1^3 v_2^3 & -v_1^3 v_2^3 \\ u_1^4 & v_1^4 & 1 & 0 & 0 & 0 & -u_1^4 u_2^4 & -v_1^4 u_2^4 \\ 0 & 0 & 0 & u_1^4 & v_1^4 & 1 & -u_1^4 v_2^4 & -v_1^4 v_2^4 \end{pmatrix} \begin{pmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \end{pmatrix} = \begin{pmatrix} u_2^1 \\ v_2^1 \\ u_2^2 \\ v_2^2 \\ u_2^3 \\ v_2^3 \\ u_2^4 \\ v_2^4 \end{pmatrix}$$

将 H 矩阵看成向量，通过解该向量的线性方程来恢复 H 。称为**直接线性变换法（DLT）**

系数矩阵由特征点位置构成